

Instituto Politécnico Nacional — Escuela Superior de Cómputo

Ingeniería en Sistemas Computacionales / Unidad de aprendizaje: Bases de Datos

Práctica 1 — Ejercicio 4: Estado del arte, tres artículos científicos

José Luis Villanueva Morales — Boleta 2024630016

Artículo 1

Echihabi, K., Fatourou, P., Zoumpatianos, K., Palpanas, T., & Benbrahim, H. (2022). Hercules against data series similarity search. *Proceedings of the VLDB Endowment*, 15(10), 2005–2018. <https://doi.org/10.14778/3547305.3547308>

Problema que aborda: los sistemas existentes para buscar similitud entre series de datos (secuencias numéricas masivas, como señales de sensores) no logran ofrecer tiempos de respuesta interactivos cuando las colecciones son demasiado grandes para caber en memoria y deben procesarse desde disco.

Método o propuesta: los autores proponen Hercules, un índice arbóreo paralelo para búsqueda exacta de similitud sobre colecciones de series de datos almacenadas en disco. La técnica combina algoritmos de construcción de índice y de resolución de consultas que aprovechan distintas técnicas de resumen (summarization), planifican con cuidado las operaciones costosas, optimizan los accesos a memoria y disco, y explotan el multihilo y las capacidades SIMD del hardware moderno.

Resultado principal: mediante una evaluación experimental extensa contra técnicas del estado del arte, en datasets sintéticos y reales, Hercules resulta hasta un orden de magnitud más rápido que su mejor competidor, aunque este competidor no siempre es el mismo según el escenario.

Relación con la Unidad Temática I: se vincula directamente con el módulo del procesador de consultas y con la clasificación de los SGBD por su modelo de datos, ya que las series de tiempo son un tipo de dato que exige estructuras de indexación distintas a las de un SGBD relacional convencional.

Aporte para el proyecto del curso: ilustra que el diseño de índices no es genérico, sino que depende del tipo de dato y del patrón de consulta; al modelar mi caso de estudio conviene pensar, desde el diseño conceptual, en qué consultas serán frecuentes para anticipar qué índices tendrán sentido en la fase de diseño físico.

Artículo 2

Wu, S., Cui, Y., Yu, J., Sun, X., Kuo, T.-W., & Xue, C. J. (2022). NFL: Robust learned index via distribution transformation. *Proceedings of the VLDB Endowment*, 15(10), 2188–2200. <https://doi.org/10.14778/3547305.3547322>

Problema que aborda: los índices basados en aprendizaje automático (learned indexes) aproximan la función que mapea llaves a posiciones mediante funciones lineales por tramos; cuando la distribución de las llaves es muy irregular, la segmentación necesaria para lograr una buena aproximación introduce una sobrecarga que perjudica el rendimiento.

Método o propuesta: los autores proponen NFL, un marco de dos etapas que primero aplica una transformación de distribución (una "Numerical Normalizing Flow") para convertir la distribución original de las llaves en una distribución casi uniforme, y después construye, sobre esas llaves transformadas, un índice robusto llamado After-Flow Learned Index (AFLI).

Resultado principal: en evaluaciones con cargas de trabajo sintéticas y reales, NFL logra el mayor throughput (rendimiento) y la menor latencia de cola frente a los índices aprendidos más avanzados existentes al momento de la publicación.

Relación con la Unidad Temática I: se relaciona con los modelos de datos y con los módulos internos de un SGBD (particularmente el procesador de consultas y el gestor de almacenamiento), mostrando una alternativa emergente a los índices clásicos (B-Tree, hash) que suelen enseñarse como referencia.

Aporte para el proyecto del curso: refuerza la idea de que la elección de estructuras de acceso no es un detalle menor de implementación, sino una decisión de diseño que depende de cómo se distribuyen realmente los datos del negocio que se está modelando.

Artículo 3

Namdeo, B., & Suman, U. (2021). Schema design advisor model for RDBMS to NoSQL database migration. *International Journal of Information Technology*, 13(1), 277–286. <https://doi.org/10.1007/s41870-020-00515-8>

Problema que aborda: migrar una base de datos relacional (RDBMS) hacia un modelo NoSQL no es trivial, porque el esquema rígido y normalizado del modelo relacional no tiene una correspondencia directa con las estructuras flexibles y desnormalizadas típicas de NoSQL; elegir un mal diseño de esquema NoSQL puede degradar el rendimiento en vez de mejorarlo.

Método o propuesta: los autores desarrollan un modelo asesor ("schema design advisor") que, a partir del esquema relacional de origen y de un modelo de costos para NoSQL, recomienda cómo reestructurar los datos al migrar, buscando minimizar el costo de las operaciones más frecuentes en el nuevo esquema.

Resultado principal: el modelo propuesto permite tomar decisiones de diseño de esquema informadas por costos durante la reingeniería de una base de datos relacional hacia NoSQL, en lugar de dejar la decisión al criterio informal del desarrollador.

Relación con la Unidad Temática I: conecta directamente con el tema de modelos de datos y su clasificación, en particular con la distinción entre modelos basados en registros (relacional) y modelos NoSQL (documental, columnar, etc.), y con el ciclo de vida de una base de datos en su etapa de rediseño/migración.

Aporte para el proyecto del curso: aporta un criterio para justificar, en el ejercicio de modelado, por qué se elige un modelo relacional (o no) para el caso de estudio propuesto, en función del tipo de consultas y de la estructura de los datos, y no solo por familiaridad con la herramienta.

Comparación de los tres artículos

Los tres artículos comparten un mismo hilo conductor: todos abordan el problema de cómo organizar y acceder eficientemente a los datos cuando el modelo o la estructura de acceso clásicos dejan de ser suficientes, ya sea por el volumen de los datos (Hercules), por la irregularidad de su distribución (NFL) o por el cambio de modelo de datos durante una migración (Namdeo y Suman). Difieren, sin embargo, en el nivel en el que intervienen: los dos primeros trabajan a nivel de estructura de índice dentro de un motor de bases de datos, mientras que el tercero trabaja a nivel de diseño de esquema, un paso anterior en el ciclo de vida de la base de datos. A partir de esta lectura, el problema abierto que identifico es que las herramientas de asesoría de diseño —como la de Namdeo y Suman— rara vez consideran, junto con la estructura del esquema, el tipo de índice más adecuado para las consultas resultantes; integrar ambas decisiones (esquema e índice) en una sola recomendación parece un área todavía poco explorada.